

SITM-MIO
INFORME – PARCIAL 3

INGENIERIA DE SOFTWARE IV
INGENIERO DE SISTEMAS ALEJANDRO MUÑOZ BRAVO

DAVID VERGARA LAVERDE - A00402237
KAROLD MEJIA - A00401806

01/12/2025
UNIVERSIDAD ICESI
CALI, VALLE DEL CAUCA

Tabla de contenido

Introducción	2
Nivel 1: Subsistemas	3
Nivel 2: Componentes por Subsistema	3
Formatos bicolunnares.....	4
CU-ESC-01: Procesar Datos Históricos y Calcular Velocidad Promedio por Arcos	4
CU-PROC-01: Identificar Arco de Posición GPS	5
CU-PROC-02: Calcular Velocidad Promedio por Arco	7
CU-PUB-01: Consultar Velocidad Promedio por Arco (API REST).....	9

Introducción

El SITM-MIO de Cali transporta aproximadamente 450 mil pasajeros diarios con una flota de 1000 buses. Para mejorar la calidad del servicio, se requiere un sistema automatizado que supervise el estado del transporte en tiempo real y proporcione información útil tanto a los controladores de operación como a los ciudadanos.

Este documento presenta el análisis arquitectónico del sistema enfocado en el escenario 1, que corresponde al procesamiento de datos históricos para calcular velocidades promedio por arco. El análisis incluye el particionamiento del sistema en

subsistemas y componentes, la especificación detallada de los casos de uso principales mediante formatos bicolumnares, el diagrama de deployment que muestra la arquitectura física con sus atributos de calidad y el informe de experimentos y ejecución.

El sistema procesa entre 2.5 y 3 millones de eventos diarios, permitiendo que 40 controladores supervisen sus zonas asignadas mediante un dashboard operativo, mientras que los ciudadanos pueden consultar el estado del transporte a través de una API REST pública.

Particionamiento



Nivel 1: Subsistemas

Subsistema	Descripción
Subsistema de Ingesta de Datos	Encargado de la recolección y distribución de los datagramas
Subsistema de Procesamiento	Responsable del calculo de velocidades y generación de indicadores
Subsistema de Almacenamiento	Gestiona la persistencia de los datos y resultados
Subsistema de Publicación	Expone los resultados a usuarios externos y controladores

Nivel 2: Componentes por Subsistema

Subsistema de Ingesta de Datos

- DataCollector
- KafkaBroker
- StreamProcessor

Subsistema de Procesamiento

- CalculadorIndicadores
- ArcIdentifier
- ReportesIndicadores

Subsistema de Almacenamiento

- DataDB
- ReportesDB
- EstructuraVialDB
- TempReportDB

Subsistema de Publicación

- API REST pública (UserController)
- Dashboard Controladores (UICCO)

Formatos bicolumnares

CU-ESC-01: Procesar Datos Históricos y Calcular Velocidad Promedio por Arcos

ID	CU-ESC-01
Autor	David Vergara
Caso de uso	Procesar Datos Históricos y Calcular Velocidad Promedio por Arcos
Casos de uso relacionados	CU-PROC-01, CU-PROC-02, CU-PUB-01
Actor	Sistema de Análisis (CalculadorIndicadores)
Precondición	- Datagramas GPS históricos de 1 año están en DataDB

	- Estructura de rutas y arcos definida en EstructuraVialDB - DeviceAnálisis operativo
Contexto	El sistema procesa automáticamente datos históricos de un año para calcular velocidades promedio por arco y publicar resultados.
Acciones del usuario	Acciones del sistema (Incluye flujo normal y excepciones)
	1. El sistema inicia el proceso de análisis histórico programado
	2. El sistema consulta total de datagramas en DataDB
	3. El sistema calcula nodos requeridos y distribuye datagramas en lotes
	4. El sistema ejecuta CU-PROC-01: Identificar Arco de Posición GPS en paralelo
	5. El sistema ejecuta CU-PROC-02: Calcular Velocidad Promedio por Arco en paralelo
	6. El sistema persiste resultados en ReportesDB
	7. El sistema marca resultados como "disponibles para consulta"
	8. El sistema habilita la consulta de resultados mediante API REST (CU-PUB-01)
Poscondición	- Velocidades promedio calculadas para todos los arcos - Resultados disponibles en API REST
Observaciones	N/A

CU-PROC-01: Identificar Arco de Posición GPS

ID	CU-PROC-01
----	------------

Autor	David Vergara
Caso de uso	Identificar Arco de Posición GPS
Casos de uso relacionados	CU-ESC-01, CU-PROC-02
Actor	ArcIdentifier (DeviceAnalysis)
Precondición	- Datagramas históricos en DataDB - Estructura de arcos con geometría en EstructuraVialDB
Contexto	El componente mapea cada posición GPS a su arco correspondiente usando geometría espacial para permitir cálculo de velocidades.
Acciones del usuario	Acciones del sistema (Incluye flujo normal y excepciones)
	1. El sistema recibe datagramas desde DeviceIngesta
	2. El sistema agrupa datagramas por id_ruta
	3. El sistema carga geometría de arcos desde EstructuraVialDB en memoria
	4. Por cada datagrama, extrae: id_bus, timestamp, latitud, longitud, id_ruta
	5. El sistema calcula distancia del punto GPS a cada arco de la ruta
	6. El sistema identifica arco más cercano (distancia mínima)
	7. El sistema determina dirección (normal/inverso) comparando con posición anterior

	8. El sistema enriquece datagrama con: id_arco, direccion, distancia_al_arco
	9. El sistema persiste en TempReportDB
	10. El sistema agrupa por id_arco y ordena por (id_bus, timestamp)
	11. El sistema envía datagramas agrupados a CU-PROC-02
Poscondición	- Cada datagrama tiene id_arco asignado - Datagramas agrupados por arco y ordenados - Listos para cálculo de velocidades
Observaciones	N/A

CU-PROC-02: Calcular Velocidad Promedio por Arco

ID	CU-PROC-02
Autor	David Vergara
Caso de uso	Calcular Velocidad Promedio por Arco
Casos de uso relacionados	CU-PROC-01, CU-PUB-01
Actor	CalculadorIndicadores (DeviceAnalysis)
Precondición	- Datagramas enriquecidos con id_arco en TempReportDB - Agrupados por arco y ordenados - Distancias de arcos en EstructuraVialDB
Contexto	El componente calcula velocidades promedio, estadísticas y clasificación de estado por cada arco usando datos históricos.

Acciones del usuario	Acciones del sistema (Incluye flujo normal y excepciones)
	1. El sistema recibe datagramas agrupados por id_arco
	2. Por cada arco, consulta distancia_metros desde EstructuraVialDB
	3. El sistema identifica parejas consecutivas del mismo bus en el arco
	4. Por cada pareja calcula: tiempo_transito = timestamp_salida - timestamp_entrada
	5. El sistema agrega tiempos válidos a lista
	[Caso: Datos insuficientes] 6. Si menos de 30 muestras: - Marca arco "sin_datos_suficientes" - Asigna estado "desconocido" - Continúa con siguiente arco
	7. El sistema calcula: tiempo_promedio
	8. El sistema calcula: velocidad_promedio_kmh
	9. El sistema clasifica estado del arco: - velocidad ≥ 40 km/h: "fluido" - $20 \leq \text{velocidad} < 40$: "lento" - velocidad < 20 : "congestionado"
	10. El sistema persiste resultados en ReportesDB
	11. El sistema marca resultados como "disponibles_para_consulta"
Poscondición	- Velocidades calculadas y almacenadas en ReportesDB - Sistema listo para consultas API REST

	- Dashboard puede consumir información
Observaciones	N/A

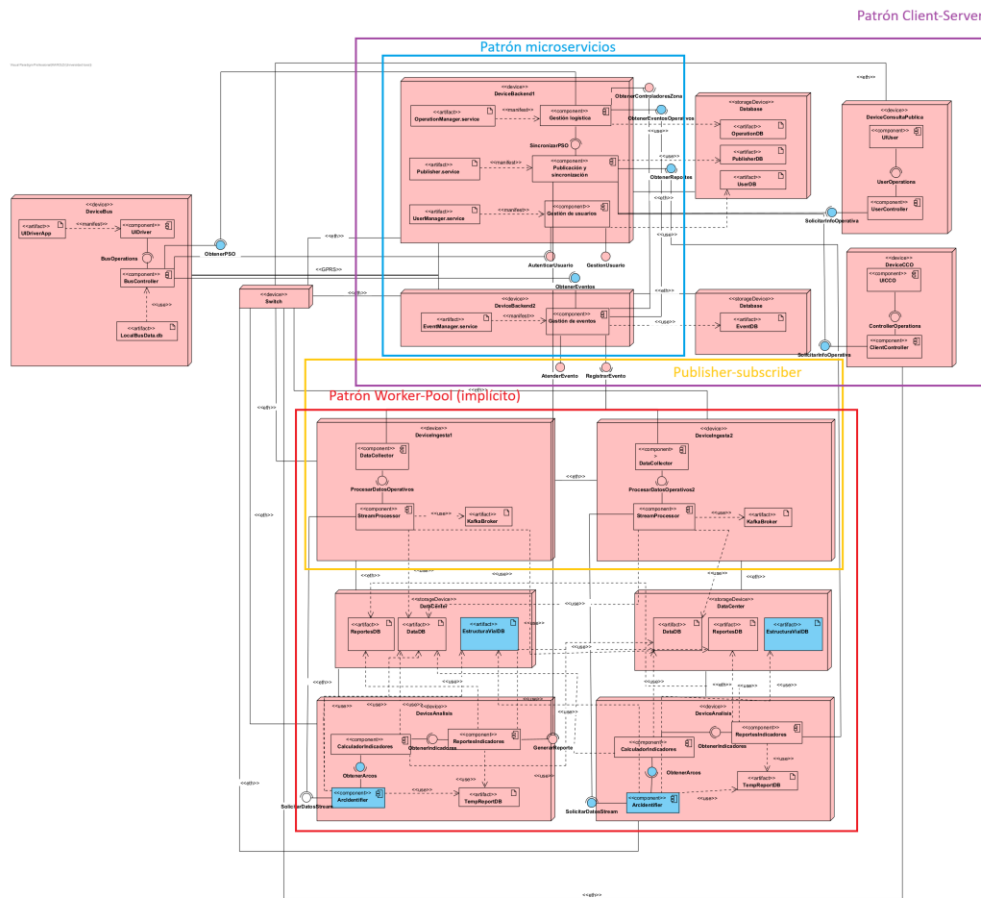
CU-PUB-01: Consultar Velocidad Promedio por Arco (API REST)

ID	CU-PUB-01
Autor	David Vergara
Caso de uso	Consultar Velocidad Promedio por Arco (API REST)
Casos de uso relacionados	CU-PROC-02
Actor	Ciudadano / Sistema Externo
Precondición	- Velocidades calculadas en ReportesDB - API REST (UserController) desplegada en DeviceConsultaPublica
Contexto	Permite a ciudadanos y entidades consultar velocidades promedio y tiempos de viaje mediante servicios REST interoperables.
Acciones del usuario	Acciones del sistema (Incluye flujo normal y excepciones)
1. El actor envía GET /api/v1/arcos/{id_arco}/velocidad	
	2. El sistema valida formato de id_arco (numérico)
	3. El sistema consulta ReportesDB
	[Caso: No encontrado] 4. Si no existe: - Retorna HTTP 404 - {"error": "Arco no encontrado"}

	5. El sistema formatea respuesta JSON: - id_arco, distancia_metros - velocidad_promedio_kmh - tiempo_promedio_minutos - estado, factor_congesion - ultima_actualizacion
	6. El sistema retorna HTTP 200 OK
	7. El sistema registra métrica: timestamp, ip_cliente, tiempo_respuesta
Caso alternativo: Tiempo de viaje entre puntos	
9. El actor envía POST /api/v1/viajes/origen-destino, id_ruta	
	10. El sistema valida coordenadas
	11. El sistema consulta EstructuraVialDB para identificar arcos en ruta
	12. El sistema suma tiempos promedio de arcos
	13. El sistema retorna: tiempo_estimado, distancia_total, arcos_incluidos, estado
Poscondición	- Cliente recibe información solicitada - Consulta registrada en métricas
Observaciones	API pública sin autenticación obligatoria

Diagrama de deployment

El diagrama de deployment implementa varios atributos de calidad mediante decisiones arquitectónicas prácticas. A continuación, se describen los atributos presentes y cómo se logran:



Patrones reflejados en el diagrama:

1. **Microservicios:** El sistema sigue una arquitectura de microservicios donde cada funcionalidad de negocio (Gestión logística, Gestión de usuarios, Gestión de eventos) se implementa como servicios independientes desplegados en DeviceBackend1 y DeviceBackend2. Cada servicio tiene su propia base de datos dedicada (OperationDB, UserDB, EventDB), permitiendo desarrollo, despliegue y escalado independiente. Los servicios se comunican mediante APIs y mensajes a través de KafkaBroker.
2. **Client-server:** La arquitectura cliente-servidor separa claramente los roles: dispositivos cliente como DeviceCCO (Centro de Control Operativo) y DeviceConsultaPublica solicitan servicios a servidores backend (DeviceBackend1/2). El BusController actúa como intermediario, gestionando solicitudes como ObtenerControladoresZona y AutenticarUsuario. La comunicación sigue un modelo de petición-respuesta a través de Ethernet/GPRS.

3. **Producer-subscriber:** Patrón de mensajería desacoplado donde Publisher.service publica eventos (ej: SincronizarPSO) a KafkaBroker, y múltiples componentes suscritos procesan estos mensajes de forma independiente y simultánea.
4. **Worker-pool (implícito):** Aunque no explícito en el diagrama, se infiere un patrón worker-pool en el procesamiento concurrente: múltiples instancias de StreamProcessor consumen particiones de Kafka en paralelo formando un pool de workers. Similarmente, CalculadorIndicadores y ReportesIndicadores en DeviceAnálisis procesan múltiples solicitudes de indicadores y generación de reportes concurrentemente, utilizando TempReportDB para resultados intermedios. Esto permite escalabilidad horizontal y procesamiento paralelo de tareas.

Escalabilidad

El sistema implementa escalabilidad mediante procesamiento distribuido en el nodo DeviceAnálisis. Los componentes ArcIdentifier y CalculadorIndicadores procesan lotes de datos históricos en paralelo, permitiendo agregar nodos adicionales según el volumen de datagramas a procesar.

Esta decisión arquitectónica permite escalar horizontalmente el procesamiento batch sin modificar el código, simplemente agregando más instancias de DeviceAnálisis cuando el volumen de datos históricos aumenta. Aunque no implementa formalmente el patrón "Worker Pool", utiliza el mismo principio: distribuir trabajo entre múltiples procesadores independientes.

Throughput

KafkaBroker actúa como buffer intermedio entre la ingesta de datos (StreamProcessor) y el procesamiento (DataDB), permitiendo desacoplar la velocidad de producción de eventos de la velocidad de consumo. Esto permite al sistema manejar el alto volumen de eventos diarios (2.5-3 millones) sin que los picos de tráfico afecten el procesamiento.

Performance

El sistema utiliza bases de datos especializadas para diferentes tipos de información, evitando joins complejos y optimizando las consultas:

- DataDB: Almacena datagramas históricos (datos crudos)

- EstructuraVialDB: Contiene información maestra de rutas y arcos con geometría
- TempReportDB: Buffer temporal para resultados intermedios
- ReportesDB: Resultados finales pre-calculados

Esta separación permite que cada base de datos tenga índices optimizados para su tipo de consulta específica, reduciendo tiempos de respuesta. Por ejemplo, EstructuraVialDB utiliza índices espaciales para consultas geométricas, mientras que ReportesDB usa índices simples por id_arco para consultas de API REST.

Modularidad

El sistema separa claramente las responsabilidades entre devices:

- DeviceIngesta1/2: Recolección y validación de datos
- DeviceAnalysis: Procesamiento analítico y cálculos
- DataCenter: Persistencia especializada

Esta separación facilita el mantenimiento y la modificabilidad, ya que cada device puede actualizarse o escalarse independientemente sin afectar a los demás. Aunque no implementa formalmente el patrón "Layered Architecture" o "Microservices", sigue el mismo principio de separación de concerns.

Implementación

Para la implementación, tomando en cuenta que el sistema se centra en la ingesta y procesamiento de los datos, más específicamente el cálculo de las velocidades entre arcos, nos centramos en el patrón implícito Worker Pool, que como fue descrito anteriormente, se centra en la subdivisión de los eventos que llegan, para que puedan ser procesados por múltiples nodos. El mapeo de los componentes en el diagrama y las clases más importantes que fueron implementadas es:

Componente Diagrama	Clase Principal	Rol en Worker Pool
DeviceBackend (Master)	SpeedCalculatorMasterImpl	Coordinador - Distribuye tareas
DeviceIngesta	DatagramReaderService	Cola de Tareas - Crea batches

DeviceAnalysis (Workers)	SpeedCalculatorWorkerImpl	Workers - Procesan batches
ThreadPool (implícito)	ExecutorService	Dispatcher - Gestiona threads
KafkaBroker	Batches en memoria	Cola intermedia